

Clock–Phase–Motor Architecture with IMU-Driven PPO for Hexapod Gait Parameter Tuning

ODE-Based Simulation and Reinforcement Learning Design Document

E90 Hexapod Team

April 8, 2026

Contents

1	Introduction and Motivation	4
1.1	Why This Step Matters	4
1.2	Scope of This Document	4
2	System Architecture	5
2.1	High-Level Data Flow	5
2.2	Layer Descriptions	5
3	Clock–Phase–Motor Formulation	6
3.1	Gait Clock	6
3.2	Tripod Phase Mapping	6
3.3	Sinusoidal Motor Functions	7
3.4	Reflex Coupling	7
4	Body ODE with IMU Signals	8
4.1	State Vector	8
4.2	Equations of Motion	8
4.2.1	Contact Model	9
4.2.2	Vertical Forces	9
4.2.3	Thrust Forces	9

4.2.4	Roll and Pitch Torques	9
4.3	IMU Signal Extraction	10
4.4	Domain Randomization	10
5	PPO Reinforcement Learning Formulation	10
5.1	MDP Definition	10
5.2	Observation Space	11
5.3	Action Space	11
5.4	Reward Function	12
5.4.1	IMU Stability Reward	12
5.4.2	Forward Velocity Reward	12
5.4.3	Effort Penalty	13
5.4.4	Alive Bonus and Fall Penalty	13
5.5	PPO Hyperparameters	13
6	Gymnasium Environment Specification	14
6.1	Episode Structure	14
6.2	Integration Details	14
7	Feasibility Analysis	15
7.1	What Is Highly Feasible	15
7.2	What Requires Careful Tuning	15
7.3	Risks and Mitigations	16
7.4	Baseline Comparisons	16
8	Implementation Plan	17
8.1	File Specifications	17
8.2	Dependencies	18
9	Connection to Broader Project	19
9.1	Relationship to Existing Components	19
9.2	Upgrade Path	19

1 Introduction and Motivation

This document presents the detailed design for a **lightweight, ODE-based hexapod gait simulator** integrated with **Proximal Policy Optimization (PPO)** to tune gait architecture parameters using **IMU feedback**. The system is intended to fill the currently empty `ode_optimization_with_imu/` folder and extend the existing open-loop no-IMU optimizer into a closed-loop learning system.

1.1 Why This Step Matters

The project roadmap defines a progression from open-loop gait optimization to closed-loop adaptive control:

1. **No-IMU ODE optimization** (completed): SciPy SLSQP tunes 25 gait parameters against a reduced body ODE. The optimizer minimizes roll/pitch RMS, frequency tracking error, and servo overuse. There is no feedback during the simulated gait—parameters are fixed for each trial.
2. **IMU-driven ODE optimization with PPO** (this document): The gait runs in a body ODE that produces IMU-like signals (roll, pitch, angular rates). A PPO agent observes these signals at each gait window and adjusts a compact set of architecture parameters (reflex gains, stride/height/frequency scaling). The agent learns *when* and *how much* to adjust parameters in response to body state.
3. **Full MuJoCo PPO** (future): Transfer lessons from the lightweight ODE+PPO to the full rigid-body MuJoCo hexapod environment already implemented in `e90_repo/`.
4. **Hardware deployment** (future): Export the trained parameter-tuning policy to Teensy firmware, receiving real BNO055 IMU data.

The lightweight ODE simulator is fast (approximately 5 ms per gait cycle on a single CPU core), enabling rapid PPO iteration without the overhead of MuJoCo. This makes it the ideal testbed for validating the clock-phase-motor-PPO architecture before committing to heavier simulation or hardware.

1.2 Scope of This Document

This report covers:

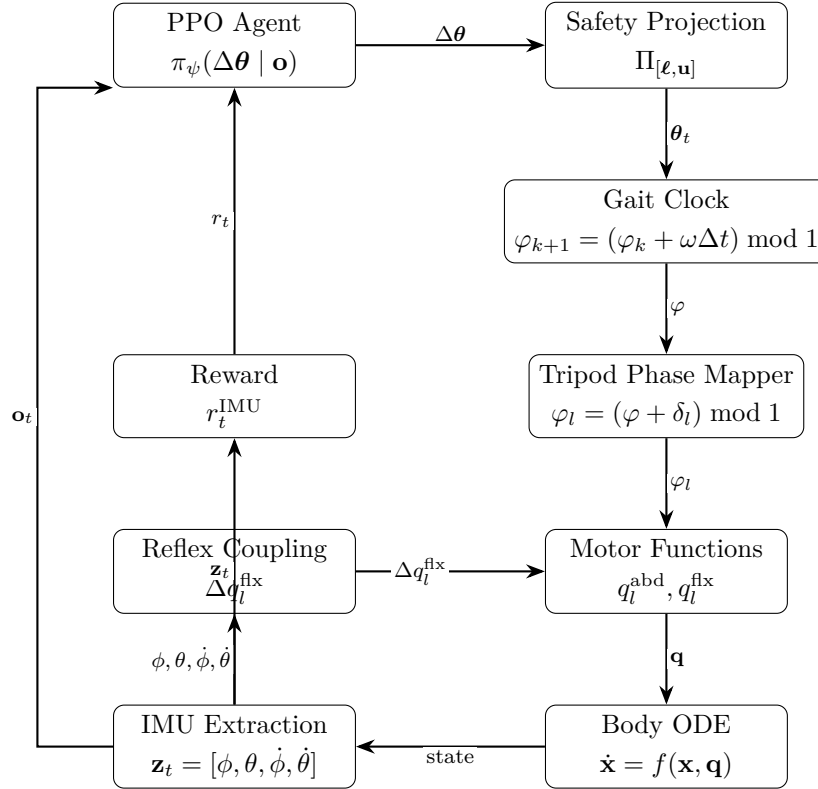
- The clock-phase-motor architecture and its mathematical formulation
- The extended body ODE with IMU signal extraction and reflex coupling

- The Gymnasium environment wrapping the ODE as an RL problem
- PPO formulation: observation space, action space, reward function, and hyperparameters
- Feasibility analysis and risk assessment
- Implementation plan with file-by-file specification
- Relationship to existing codebase components

2 System Architecture

2.1 High-Level Data Flow

The system has five interconnected layers. At each reinforcement learning step, data flows as follows:



2.2 Layer Descriptions

PPO Agent: A Stable-Baselines3 PPO policy network. Observes IMU state and gait clock phase. Outputs bounded parameter deltas $\Delta \theta \in \mathbb{R}^7$.

Safety Projection: Applies box constraints and slew-rate limits to ensure parameters remain in the safe operating region: $\boldsymbol{\theta}_{t+1} = \Pi_{[\ell, \mathbf{u}]}(\boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}_t)$.

Gait Clock: A phase oscillator $\varphi \in [0, 1)$ advancing at angular frequency $\omega = s_{\text{freq}} \cdot \omega_0$. This is the single timing source for the entire gait.

Tripod Phase Mapper: Splits the global clock into per-leg local phases with tripod offsets: $\delta_l = 0$ for Tripod A (FL, MR, RL) and $\delta_l = 0.5$ for Tripod B (FR, ML, RR).

Motor Functions: Maps each leg’s local phase to abduction and flexion servo commands using sinusoidal waveforms with per-leg rest angles, amplitudes, and phase shifts.

Reflex Coupling: IMU-derived roll and pitch corrections are distributed to each leg’s flexion command based on the leg’s side (s_l) and front-rear position (r_l).

Body ODE: An 8-state reduced body model solved with `scipy.integrate.solve_ivp`. States: $[x, \dot{x}, z, \dot{z}, \phi, \dot{\phi}, \theta, \dot{\theta}]$ (longitudinal position, vertical height, roll, pitch).

IMU Extraction: Reads roll ϕ , pitch θ , and their rates $\dot{\phi}, \dot{\theta}$ from the ODE state. Computes angular accelerations $\ddot{\phi}, \ddot{\theta}$ via finite differences. These are the signals a real BNO055 IMU would provide.

Reward: Evaluates gait quality from IMU signals using an exponential stability metric plus forward velocity bonus and effort penalty.

3 Clock–Phase–Motor Formulation

3.1 Gait Clock

Define a normalized global gait phase $\varphi \in [0, 1)$ that advances each simulation tick by:

$$\varphi_{k+1} = (\varphi_k + \omega_k \Delta t) \bmod 1 \quad (1)$$

where $\omega_k = s_{\text{freq}} \cdot \omega_0$ is the instantaneous gait frequency (Hz), s_{freq} is a PPO-tunable frequency scaling factor, and ω_0 is the baseline frequency from the no-IMU optimization (approximately 1.8 Hz).

3.2 Tripod Phase Mapping

The hexapod has six legs partitioned into two tripod groups:

$$\mathcal{A} = \{\text{FL}, \text{MR}, \text{RL}\} \quad (\text{Tripod A}) \quad (2)$$

$$\mathcal{B} = \{\text{FR}, \text{ML}, \text{RR}\} \quad (\text{Tripod B}) \quad (3)$$

Per-leg local phase with anti-phase coupling:

$$\varphi_l = (\varphi + \delta_l) \bmod 1, \quad \delta_l = \begin{cases} 0.0 & l \in \mathcal{A} \\ 0.5 & l \in \mathcal{B} \end{cases} \quad (4)$$

This ensures that when Tripod A legs are in swing phase ($\varphi_l \approx 0.25$), Tripod B legs are in stance/push phase ($\varphi_l \approx 0.75$), and vice versa.

3.3 Sinusoidal Motor Functions

For each leg l (indexed $i = 0, \dots, 5$ matching `LEG_ORDER = [FL, FR, ML, MR, RL, RR]`), the servo commands are sinusoidal functions of the local phase:

$$q_l^{\text{abd}}(t) = \beta_l^{\text{abd}} + s_{\text{stride}} \cdot A_l^{\text{abd}} \cdot \sin(2\pi\varphi_l + \psi_l^{\text{abd}}) \quad (5)$$

$$q_l^{\text{flx}}(t) = \beta_l^{\text{flx}} + s_{\text{height}} \cdot A_l^{\text{flx}} \cdot \sin(2\pi\varphi_l + \psi_l^{\text{flx}}) \quad (6)$$

where:

- $\beta_l^{\text{abd}}, \beta_l^{\text{flx}}$: per-leg rest angles (radians), from no-IMU optimization
- $A_l^{\text{abd}}, A_l^{\text{flx}}$: per-leg amplitudes (radians), from no-IMU optimization defaults
- $\psi_l^{\text{abd}}, \psi_l^{\text{flx}}$: per-leg phase shifts (radians), from no-IMU optimization
- s_{stride} : stride amplitude scale (PPO-tunable, dimensionless)
- s_{height} : lift height scale (PPO-tunable, dimensionless)

The tripod phase offset δ_l is already embedded in φ_l via Equation (4), plus the existing per-servo phase shifts from the original no-IMU `TRIPOD_PHASE` dictionary (0 for Tripod A, π for Tripod B) ensure anti-phase coupling.

3.4 Reflex Coupling

IMU measurements of roll ϕ and pitch θ are fed back to each leg's flexion command. Define:

$$\Delta_{\text{roll}} = K_{\phi} \phi + K_{\dot{\phi}} \dot{\phi} \quad (7)$$

$$\Delta_{\text{pitch}} = K_{\theta} \theta + K_{\dot{\theta}} \dot{\theta} \quad (8)$$

where $K_{\phi}, K_{\dot{\phi}}, K_{\theta}, K_{\dot{\theta}}$ are reflex gains (PPO-tunable).

Per-leg flexion correction:

$$\Delta q_l^{\text{flx}} = s_l \cdot \Delta_{\text{roll}} + r_l \cdot \Delta_{\text{pitch}} \quad (9)$$

with coupling signs:

	FL	FR	ML	MR	RL	RR
Side sign s_l	+1	-1	+1	-1	+1	-1
Front-rear sign r_l	-1	-1	0	0	+1	+1

Physical interpretation:

- s_l : when the body rolls left ($\phi > 0$), left legs extend (push down) and right legs retract to level the body.
- r_l : when the body pitches nose-down ($\theta > 0$), rear legs extend and front legs retract to level the body. Middle legs are unaffected by pitch correction.

Final flexion command with reflex:

$$\hat{q}_l^{\text{flx}} = q_l^{\text{flx}} + \Delta q_l^{\text{flx}} \quad (10)$$

4 Body ODE with IMU Signals

4.1 State Vector

The reduced body model tracks eight state variables:

$$\mathbf{x} = [x, \dot{x}, z, \dot{z}, \phi, \dot{\phi}, \theta, \dot{\theta}]^\top \quad (11)$$

where x is longitudinal (forward) position, z is vertical (height), ϕ is roll angle, and θ is pitch angle. All positions are in meters and angles in radians.

4.2 Equations of Motion

The body ODE computes forces from leg contact and thrust:

$$\ddot{x} = \frac{F_{\text{thrust}} - c_x \dot{x}}{m} \quad (12)$$

$$\ddot{z} = \frac{F_{\text{vertical}} - m g - c_z \dot{z}}{m} \quad (13)$$

$$\ddot{\phi} = \frac{\tau_{\text{roll}}}{I_\phi} \quad (14)$$

$$\ddot{\theta} = \frac{\tau_{\text{pitch}}}{I_\theta} \quad (15)$$

where:

- m : total robot mass (chassis + battery + 12 servos ≈ 0.46 kg)
- $g = 9.81 \text{ m/s}^2$: gravitational acceleration
- c_x, c_z : longitudinal and vertical damping coefficients
- I_ϕ, I_θ : roll and pitch moments of inertia

4.2.1 Contact Model

Foot contact is modeled as a smooth sigmoid function of flexion angle:

$$w_l = \frac{1}{1 + \exp(-\kappa \cdot (\hat{q}_l^{\text{flx}} - \beta_l^{\text{flx}})/A_l^{\text{flx}})} \quad (16)$$

where κ is contact sharpness (from `RobotSpec.contact_sharpness` ≈ 5.5). When the leg is flexed below rest, $w_l \rightarrow 0$ (lifted); when extended, $w_l \rightarrow 1$ (in contact).

4.2.2 Vertical Forces

Per-leg vertical force:

$$F_l^z = w_l \cdot \left(\frac{mg}{N_{\text{active}}} + k_s(\hat{q}_l^{\text{flx}} - \beta_l^{\text{flx}}) - \frac{c_s}{N_{\text{active}}} \dot{z} \right) \quad (17)$$

where k_s is support gain, c_s is support damping, and $N_{\text{active}} = \max(\sum_l w_l, 1)$.

4.2.3 Thrust Forces

Forward thrust from abduction velocity:

$$F_l^x = -w_l \cdot k_{\text{stride}} \cdot \dot{q}_l^{\text{abd}} \quad (18)$$

4.2.4 Roll and Pitch Torques

$$\tau_{\text{roll}} = \sum_l y_l \cdot F_l^z - c_\phi \dot{\phi} \quad (19)$$

$$\tau_{\text{pitch}} = \sum_l x_l \cdot F_l^z - c_\theta \dot{\theta} \quad (20)$$

where x_l, y_l are the body-frame coordinates of each leg's hip attachment point.

4.3 IMU Signal Extraction

From the ODE state at each integration step, the IMU signal vector is:

$$\mathbf{z}_k = [\phi_k, \theta_k, \dot{\phi}_k, \dot{\theta}_k, \ddot{\phi}_k, \ddot{\theta}_k]^\top \quad (21)$$

where $\ddot{\phi}_k$ and $\ddot{\theta}_k$ are obtained from the ODE right-hand side (Equations (14)–(15)) or by finite differences of $\dot{\phi}, \dot{\theta}$.

This matches what a real BNO055 IMU mounted on the hexapod torso would provide:

ODE state	BNO055 output	Noise model
ϕ, θ	Euler angles (fusion mode)	$\sigma_{\text{euler}} \approx 1^\circ$
$\dot{\phi}, \dot{\theta}$	Gyroscope X/Y	$\sigma_{\text{gyro}} \approx 0.01 \text{ rad/s}$
$\ddot{\phi}, \ddot{\theta}$	Finite diff of gyro	Amplifies noise

4.4 Domain Randomization

To reduce overfitting to the simplified ODE model and improve sim-to-real transfer, the following parameters are randomized at each episode reset:

Parameter	Nominal	Range	Distribution
Total mass m (kg)	0.458	[0.39, 0.53]	Uniform
Contact sharpness κ	5.5	[4.0, 7.0]	Uniform
Support gain k_s (N/rad)	32.0	[24.0, 40.0]	Uniform
Support damping c_s (N/(m/s))	7.5	[5.0, 10.0]	Uniform
Roll damping c_ϕ	0.06	[0.04, 0.08]	Uniform
Pitch damping c_θ	0.07	[0.05, 0.09]	Uniform
IMU noise $\sigma_\phi, \sigma_\theta$ (rad)	0.0	[0, 0.02]	Uniform
IMU noise $\sigma_{\dot{\phi}}, \sigma_{\dot{\theta}}$ (rad/s)	0.0	[0, 0.015]	Uniform
Initial roll perturbation (rad)	0.0	[−0.05, 0.05]	Uniform
Initial pitch perturbation (rad)	0.0	[−0.05, 0.05]	Uniform

5 PPO Reinforcement Learning Formulation

5.1 MDP Definition

The ODE-based hexapod gait tuning problem is formulated as a Markov Decision Process (MDP):

Episode: One episode consists of N_{windows} RL steps (default: 50), each covering one gait cycle of ODE simulation.

State: The environment internal state is the full ODE state vector $\mathbf{x} \in \mathbb{R}^8$ plus current architecture parameters $\boldsymbol{\theta} \in \mathbb{R}^7$.

Observation: The agent observes a subset of the state (Section 5.2).

Action: The agent outputs bounded parameter deltas $\Delta\boldsymbol{\theta} \in \mathbb{R}^7$ (Section 5.3).

Transition: The ODE is integrated for one gait cycle ($T_{\text{cycle}} = 1/\omega$ s), parameters are updated via safety projection, and the new ODE state is returned.

Reward: IMU-based stability metric computed over the gait cycle (Section 5.4).

Termination: If $|\phi|$ or $|\theta|$ exceeds a critical angle (default: 30°), the episode terminates with a fall penalty.

5.2 Observation Space

The observation vector has 10 dimensions:

$$\mathbf{o}_t = [\phi_t, \theta_t, \dot{\phi}_t, \dot{\theta}_t, \sin(2\pi\varphi_t), \cos(2\pi\varphi_t), S_{\text{rms},t}, \bar{v}_{x,t}, \tilde{t}, \hat{q}_{\text{flex,rms}}]^\top \in \mathbb{R}^{10} \quad (22)$$

Dim	Symbol	Description
0	ϕ_t	Roll angle (rad), from ODE state
1	θ_t	Pitch angle (rad), from ODE state
2	$\dot{\phi}_t$	Roll rate (rad/s), from ODE state
3	$\dot{\theta}_t$	Pitch rate (rad/s), from ODE state
4	$\sin(2\pi\varphi_t)$	Clock signal, sine component
5	$\cos(2\pi\varphi_t)$	Clock signal, cosine component
6	$S_{\text{rms},t}$	Windowed RMS stability score
7	$\bar{v}_{x,t}$	Mean forward velocity over window
8	$\tilde{t}$	Normalized step in episode $\in [0, 1]$
9	$\hat{q}_{\text{flex,rms}}$	RMS flexion command magnitude

Why sin/cos clock signals: Providing the clock phase as $[\sin(2\pi\varphi), \cos(2\pi\varphi)]$ rather than φ directly avoids the discontinuity at $\varphi = 0/1$ and gives the agent a smooth, periodic representation of where in the gait cycle the robot is. This is critical for the agent to correlate IMU disturbances with specific gait phases (e.g., learning that roll spikes occur during swing transitions).

5.3 Action Space

The agent outputs 7 continuous values, each in $[-1, 1]$, which are scaled to parameter deltas:

$$\Delta\boldsymbol{\theta}_t = \mathbf{a}_t \odot \boldsymbol{\Delta}_{\text{max}} \quad (23)$$

Dim	Parameter	Units	ℓ	$\mathbf{u}$	$\Delta_{\max}$
0	K_ϕ (roll P gain)	rad/rad	0.0	2.0	0.1
1	$K_{\dot{\phi}}$ (roll D gain)	rad/(rad/s)	0.0	1.0	0.05
2	K_θ (pitch P gain)	rad/rad	0.0	2.0	0.1
3	$K_{\dot{\theta}}$ (pitch D gain)	rad/(rad/s)	0.0	1.0	0.05
4	s_{stride} (stride scale)	–	0.3	1.5	0.05
5	s_{height} (height scale)	–	0.3	1.5	0.05
6	s_{freq} (frequency scale)	–	0.5	1.5	0.03

Safety projection:

$$\boldsymbol{\theta}_{t+1} = \text{clip}(\boldsymbol{\theta}_t + \Delta\boldsymbol{\theta}_t, \boldsymbol{\ell}, \mathbf{u}) \quad (24)$$

The $\Delta_{\max}$ values are deliberately conservative: the maximum parameter change per RL step is small enough that the gait cannot destabilize in a single step.

5.4 Reward Function

The per-step reward combines stability, locomotion, and efficiency objectives:

$$r_t = r_t^{\text{IMU}} + \alpha_v \cdot r_t^{\text{velocity}} - \alpha_e \cdot r_t^{\text{effort}} + r_{\text{alive}} - \beta_f \cdot \mathbb{I}_{\text{fall},t} \quad (25)$$

5.4.1 IMU Stability Reward

$$r_t^{\text{IMU}} = \exp(-\mathbf{z}_t^\top \mathbf{W} \mathbf{z}_t) \quad (26)$$

where $\mathbf{W} = \text{diag}(w_\phi, w_\theta, w_{\dot{\phi}}, w_{\dot{\theta}}, w_{\ddot{\phi}}, w_{\ddot{\theta}})$ is a diagonal weight matrix. Default values:

	w_ϕ	w_θ	$w_{\dot{\phi}}$	$w_{\dot{\theta}}$	$w_{\ddot{\phi}}$	$w_{\ddot{\theta}}$
Value	50.0	50.0	5.0	5.0	0.5	0.5

This gives $r_t^{\text{IMU}} \approx 1.0$ when the body is level and still, and decays toward 0 as oscillations increase. The exponential form provides a smooth gradient everywhere.

5.4.2 Forward Velocity Reward

$$r_t^{\text{velocity}} = \min(\bar{v}_{x,t}, v_{\text{cap}})/v_{\text{cap}} \quad (27)$$

where $\bar{v}_{x,t}$ is the mean forward velocity over the gait window and $v_{\text{cap}} = 0.15$ m/s caps the reward to prevent the agent from maximizing speed at the expense of stability.

5.4.3 Effort Penalty

$$r_t^{\text{effort}} = \frac{1}{12N} \sum_{k=1}^N \sum_{c=1}^{12} |u_{c,k} - u_{c,k-1}| \quad (28)$$

penalizing servo command jitter (proportional to real servo wear and power consumption).

5.4.4 Alive Bonus and Fall Penalty

$$r_{\text{alive}} = 0.1, \quad \beta_f = 10.0 \quad (29)$$

The alive bonus encourages survival; the fall penalty terminates the episode.

5.5 PPO Hyperparameters

Hyperparameter	Symbol	Value
Learning rate	α	3×10^{-4}
Clip range	ϵ	0.2
Discount factor	γ	0.99
GAE lambda	λ	0.95
Entropy coefficient	c_{ent}	0.01
Value function coefficient	c_{vf}	0.5
Max gradient norm	–	0.5
Number of epochs per update	K	10
Minibatch size	–	64
Rollout buffer size (steps per env)	T	2048
Number of parallel envs	N_{env}	8
Total training timesteps	–	2×10^6
Policy network	–	MLP [64, 64]
Value network	–	MLP [64, 64]
Activation	–	Tanh
Initial log std	–	–0.5

Rationale for key choices:

- **Entropy coefficient 0.01:** Encourages exploration in the 7D parameter space. Higher values (0.05) may prevent convergence; lower values (0.001) may lead to premature convergence on suboptimal parameters.
- **8 parallel envs:** Each env runs a fast ODE (~ 5 ms per step), so 8 envs provide diverse rollouts without CPU bottleneck. Total wall-clock time for 2M steps: approximately 20–30

minutes.

- **Tanh activation:** Matches the bounded action space better than ReLU for small, parameter-tuning networks.
- **Initial log std -0.5 (std ≈ 0.6):** Provides meaningful exploration over the $[-1, 1]$ action space while not being so large as to produce erratic parameter changes.

6 Gymnasium Environment Specification

6.1 Episode Structure

1. **Reset:** Initialize ODE state to $\mathbf{x}_0 = \mathbf{0}$ (or with random perturbation). Initialize architecture parameters to baseline $\boldsymbol{\theta}_0$ from no-IMU optimization. Randomize ODE model parameters (domain randomization).
2. **Step:**
 - (a) Receive action $\mathbf{a}_t \in [-1, 1]^7$ from PPO.
 - (b) Scale to parameter deltas and apply safety projection (Equation (24)).
 - (c) Integrate ODE for one gait window (N_{cycles} cycles, default 2).
 - (d) During integration, reflex coupling modifies flexion commands at each tick.
 - (e) Extract IMU signals from ODE trajectory.
 - (f) Compute reward from IMU signals, velocity, and effort.
 - (g) Return observation, reward, terminated, truncated.
3. **Termination:** $|\phi| > \phi_{\text{crit}}$ or $|\theta| > \theta_{\text{crit}}$ (default 30°).
4. **Truncation:** After $N_{\text{windows}} = 50$ steps (approximately 50 gait cycles ≈ 28 s at 1.8 Hz).

6.2 Integration Details

Each RL step integrates the ODE for $T_{\text{window}} = N_{\text{cycles}}/\omega$ seconds using `solve_ivp` with RK45. The integration produces a dense time series at $\Delta t_{\text{eval}} = 0.005$ s (200 Hz), from which IMU signals are sampled.

The reflex coupling (Section 3.4) operates *inside* the ODE: at each evaluation of the ODE right-hand side, the current roll and pitch are used to modify the flexion commands via Equations (7)–(10). This creates a closed-loop feedback path within the continuous-time simulation.

7 Feasibility Analysis

7.1 What Is Highly Feasible

1. **Clock–phase–motor coupling:** The sinusoidal command structure with tripod phase offsets is already proven in `hexapod_no_imu_optimization.py`. Wrapping it in the clock/phase/motor abstraction is a refactoring exercise, not a research problem.
2. **IMU signal extraction from ODE:** The body ODE already computes roll, pitch, and their rates as explicit state variables. IMU signals are a direct readout.
3. **PPO with 7D action space:** Stable-Baselines3 PPO is well-tested for continuous action spaces. A 7-dimensional parameter space is small—PPO routinely handles 12–100D action spaces. Convergence in <2M timesteps is expected.
4. **Fast ODE simulation:** A single gait cycle with RK45 takes ~ 5 ms. With 8 parallel envs and 50 steps/episode, one episode takes ~ 2.5 s. Collecting 2M timesteps (≈ 40 k episodes) takes ~ 25 minutes wall-clock.

7.2 What Requires Careful Tuning

1. **Reward weight balance:** The relative weights of r^{IMU} , r^{velocity} , and r^{effort} will determine whether the agent prioritizes stability, speed, or energy efficiency. We propose starting with the exponential IMU reward as dominant ($\sim 60\%$ of total reward magnitude) and velocity as secondary ($\sim 30\%$). Ablation over these weights is planned.
2. **Gait window length:** If the RL step covers too few cycles, the ODE may not reach steady state and the IMU signals will be noisy. If too many cycles, the agent cannot react quickly to disturbances. We start with 2 cycles per step and evaluate 1 and 4.
3. **Parameter delta bounds (Δ_{max}):** Too large allows destabilizing jumps; too small slows adaptation. The proposed values ($\sim 5\%$ of range per step) are a conservative starting point.
4. **Domain randomization range:** Too narrow leads to overfitting to the nominal ODE model. Too wide makes the problem too hard for PPO. The proposed ranges ($\pm 15\%$ of nominal) are based on typical manufacturing tolerances for hobby-grade robots.

7.3 Risks and Mitigations

Risk	Severity	Mitigation
ODE model fidelity: reduced body model may not capture real contact dynamics	Medium	Compare ODE-trained policy against MuJoCo validation. Use domain randomization. Treat ODE+PPO as pre-training, not final policy.
ODE stiffness with reflex coupling: high reflex gains may cause integration failure	Low	Monitor <code>solve_ivp</code> success flag. Cap reflex gains in bounds. Fall back to BDF solver if RK45 fails.
PPO convergence: 7D space may have local optima	Low	Use entropy bonus, multiple seeds, and compare against SPSA/CMA-ES baseline. 7D is small enough for PPO.
Reward hacking: agent exploits ODE model artifacts	Medium	Domain randomization. Sanity-check learned parameters against physical intuition. Add regularization toward baseline parameters.
Sim-to-real gap	High (for HW transfer)	This step is simulation-only. Transfer to MuJoCo first, then to hardware. ODE serves as rapid prototyping tool.

7.4 Baseline Comparisons

To validate that PPO adds value over simpler optimization, we will compare:

1. **No adaptation (baseline):** Fixed parameters from no-IMU optimization. No reflex. $\theta = \theta_0$.
2. **Reflex only:** Hand-tuned reflex gains, no PPO. $K_\phi = K_\theta = 0.5$, $K_{\dot{\phi}} = K_{\dot{\theta}} = 0.2$.
3. **SPSA optimization:** One-shot gradient-free optimization of θ using projected SPSA (same cost function as PPO reward, but no per-step adaptation).
4. **PPO:** Learned per-step adaptation of θ .

PPO should outperform SPSA because it can *adapt during the episode* in response to changing conditions, whereas SPSA finds a single fixed θ^* .

8 Implementation Plan

All files are created under `hexapod_brain_roadmap/ode_optimization_with_imu/`.

8.1 File Specifications

File	Contents
<code>config.py</code>	Dataclasses: <code>RobotSpec</code> (reuse from no-IMU), <code>GaitDefaults</code> (amplitudes, rest angles, phase shifts from no-IMU optimization), <code>ReflexConfig</code> (gain bounds, initial values), <code>RewardConfig</code> (weights for IMU, velocity, effort, alive, fall), <code>PPOConfig</code> (hyperparameters), <code>DomainRandomConfig</code> (ranges for randomization).
<code>clock_phase_motor.py</code>	Classes: <code>GaitClock</code> (Equation (1)), <code>TripodPhaseMapper</code> (Equation (4)), <code>ServoCommandGenerator</code> (Equations (5)–(6)). Functions: <code>compute_leg_commands(t, params, theta)</code> returning 12-channel servo commands with reflex coupling.
<code>imu_body_ode.py</code>	Function: <code>body_ode_with_reflex(t, state, params, theta, spec)</code> extending the no-IMU <code>body_ode</code> to use reflex-corrected flexion commands. Function: <code>simulate_gait_window(params, theta, spec, t_start, x0, dt)</code> integrating one gait window and returning time series + IMU signals.
<code>hexapod_ode_env.py</code>	Class: <code>HexapodODEEnv(gym.Env)</code> implementing <code>reset</code> , <code>step</code> , <code>reward</code> computation, <code>termination</code> check. Uses <code>imu_body_ode</code> for simulation and <code>clock_phase_motor</code> for command generation.
<code>train_ppo.py</code>	Script: create vectorized envs, instantiate SB3 PPO, configure callbacks (eval, checkpoint, TensorBoard), run training, save final model.

File	Contents
<code>evaluate.py</code>	Script: load trained PPO checkpoint, run N evaluation episodes, compute mean/std of IMU reward, forward velocity, fall rate. Export summary JSON and time-series CSV.
<code>README.md</code>	Architecture overview, dependency list, run instructions, expected outputs.

8.2 Dependencies

Package	Version
<code>numpy</code>	≥ 1.24
<code>scipy</code>	≥ 1.11
<code>gymnasium</code>	≥ 0.29
<code>stable-baselines3</code>	≥ 2.0
<code>torch</code>	≥ 2.0
<code>tensorboard</code>	≥ 2.14
<code>matplotlib</code>	≥ 3.7 (evaluation plots)

No MuJoCo dependency is required—this is a pure SciPy ODE + PyTorch/SB3 project.

9 Connection to Broader Project

9.1 Relationship to Existing Components

Existing component	Relationship
ode_optimization_no_imu	Direct predecessor. This work reuses <code>RobotSpec</code> , leg geometry constants, default gait parameters, and the body ODE structure.
e90_repo/models/ode_gait	Parallel implementation in MuJoCo. The <code>ODETripodPolicy</code> uses the same clock-phase-motor concept with $\theta_a = 2\pi ft + \phi_{\text{offset}}$. Successful parameters from the ODE+PPO can seed the MuJoCo policy's initial conditions.
e90_repo/envs/hexapod_env.py	The full MuJoCo environment already supports <code>observation_mode="imu6"</code> and phase observations. After ODE+PPO validation, the same reward function and observation structure transfer directly.
Roadmap (Sections 6–7)	This implementation realizes Completion Block D (PPO parameter tuner) in a lightweight simulation before attempting the full firmware deployment path.
Arduino firmware (v2)	The learned parameter ranges and reflex gain magnitudes will inform the safety bounds for the eventual v3 firmware.

9.2 Upgrade Path

1. **ODE + PPO** (this work): Validate architecture, reward, and PPO convergence in fast simulation.
2. **MuJoCo + PPO**: Transfer the clock-phase-motor structure and reward function to the full MuJoCo hexapod. Use ODE-trained parameters as initialization.
3. **Sim-to-real**: Export the PPO policy network weights to C (Teensy). The 7D parameter output feeds into the v3 firmware's runtime parameter struct. IMU data comes from the real BNO055.

10 Summary

This document specifies a complete system for PPO-based gait parameter tuning using:

- A **clock function** providing a global gait frequency ω
- **Phase mapping** that splits the clock into anti-phase tripod groups
- **Sinusoidal motor functions** generating per-leg abduction and flexion commands
- **Reflex coupling** from IMU roll/pitch to leg flexion corrections
- A **reduced body ODE** providing IMU-like feedback signals
- **PPO** learning to tune 7 architecture parameters per gait cycle

The 7-dimensional action space (4 reflex gains + 3 gait scales) is small enough for reliable PPO convergence, while the clock–phase–motor structure enforces tripod gait invariants that prevent the agent from producing physically meaningless motions. The ODE simulation is fast enough (~ 5 ms/cycle) to enable training in under 30 minutes on a single machine.

The primary risks are reward-weight sensitivity and ODE model fidelity. Both are mitigated by domain randomization, ablation studies against SPSA and reflex-only baselines, and the planned upgrade path to full MuJoCo validation before hardware deployment.